

IEORE4004

Optimization Models and Methods

Yaren Bilge Kaya, PhD

Columbia University
Department of Industrial Engineering and Operations Research



Outline

- Introduction to Solving Integer Programs (IPs)
 - Branch and Bound Algorithm
 - Cutting Planes Algorithm
 - Implementation of Algorithms Using Gurobi
- Common Integer Programming Problems

Introduction to Solving Integer Programs

- Branch and Bound Algorithm
- Cutting Planes Algorithm
- Implementation of Algorithms Using Gurobi

Branch and Bound Algorithm

Branch and Bound Algorithm

Let's consider this IP to explain the method:

$$\text{Maximize } Z = 5x_1 + 6x_2$$

$$\begin{aligned} \text{Subject to } \quad & x_1 + x_2 \leq 5 \\ & 4x_1 + 7x_2 \leq 28 \\ & x_1, x_2 \geq 0 \\ & x_1, x_2 \in \mathbb{Z} \end{aligned}$$

Branch and Bound Algorithm

Let's consider this IP to explain the method:

$$\text{Maximize } Z = 5x_1 + 6x_2$$

$$\begin{aligned} \text{Subject to } & x_1 + x_2 \leq 5 \\ & 4x_1 + 7x_2 \leq 28 \\ & x_1, x_2 \geq 0 \\ & x_1, x_2 \in \mathbb{Z} \end{aligned}$$

$$\text{Maximize } Z = 5x_1 + 6x_2$$

$$\begin{aligned} \text{Subject to } & x_1 + x_2 \leq 5 \\ & 4x_1 + 7x_2 \leq 28 \\ & x_1, x_2 \geq 0 \end{aligned}$$



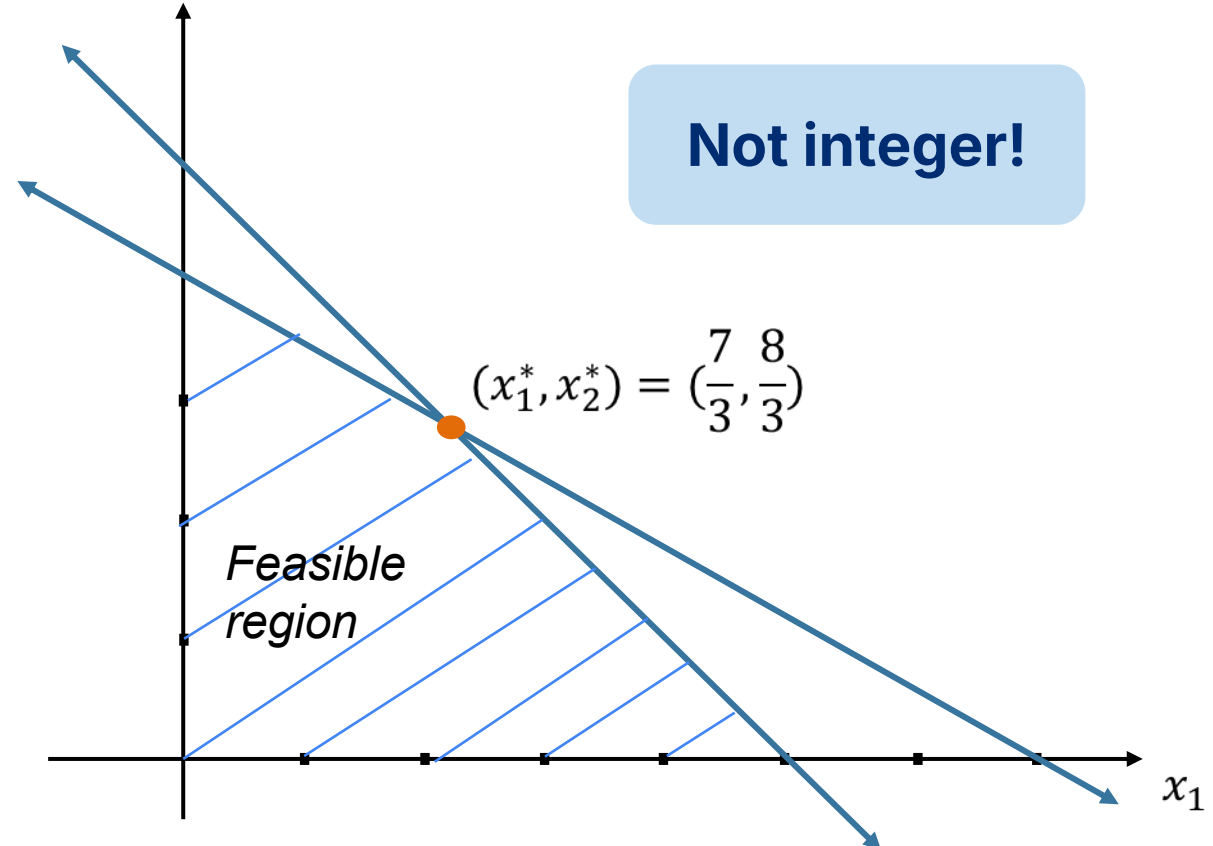
Step 1: Remove the integrality constraints, a.k.a. Linear Relaxation

Branch and Bound Algorithm

Step 2: See whether the LP's optimal solution is integer

$$\text{Maximize } Z = 5x_1 + 6x_2$$

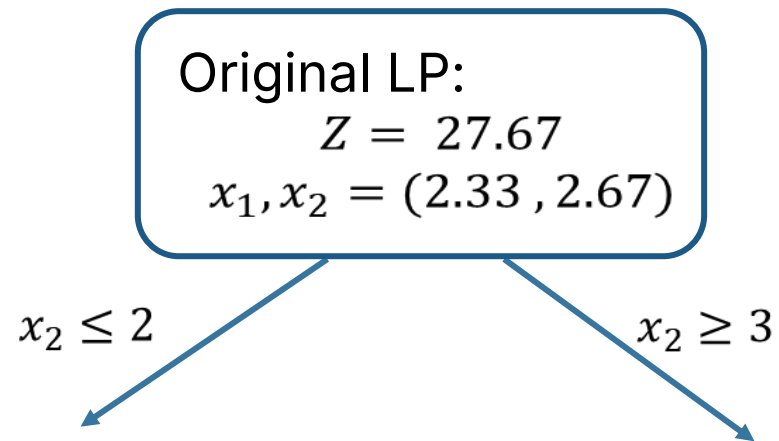
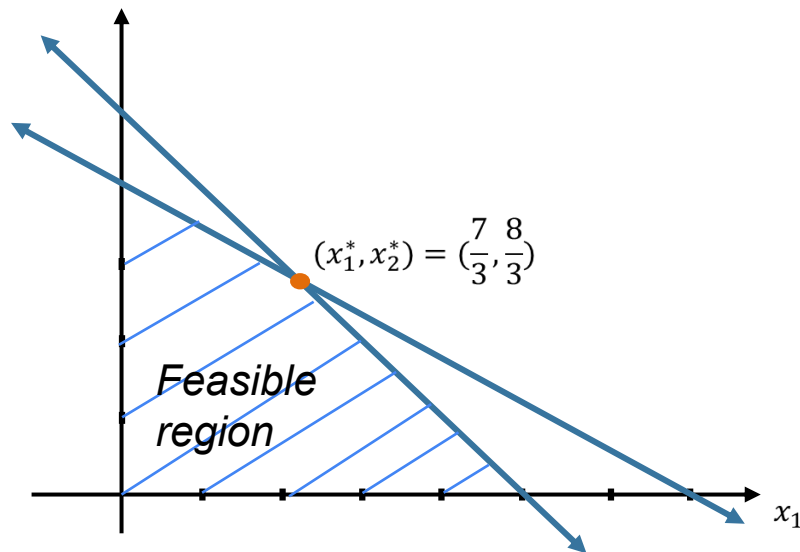
$$\begin{aligned} \text{Subject to } & x_1 + x_2 \leq 5 \\ & 4x_1 + 7x_2 \leq 28 \\ & x_1, x_2 \geq 0 \\ & x_1, x_2 \in \mathbb{Z} \end{aligned}$$



Branch and Bound Algorithm

Step 3: Check the neighboring integer solutions

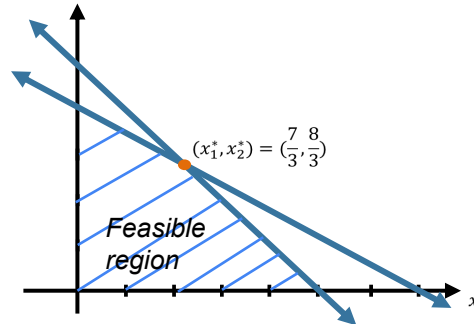
Pick randomly, or the largest decimal portion, or ...



Branch and Bound Algorithm

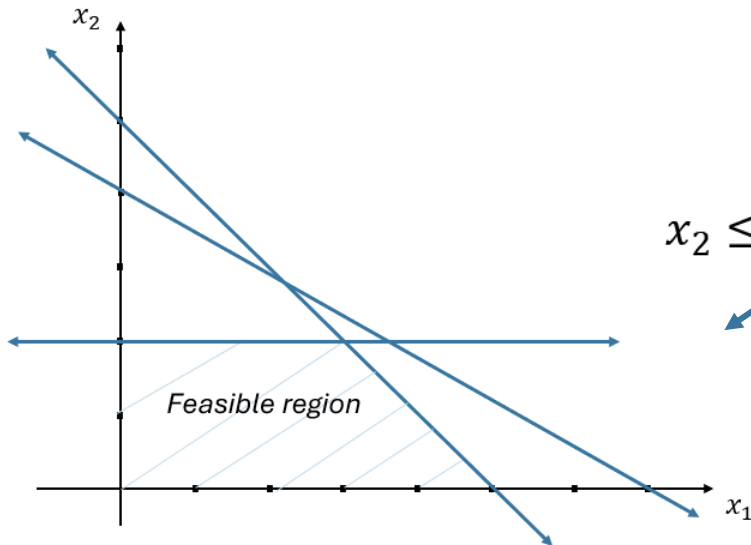
Step 3: Check the neighboring integer solutions

Maximize $Z = 5x_1 + 6x_2$
Subject to $x_1 + x_2 \leq 5$
 $4x_1 + 7x_2 \leq 28$
 $x_1, x_2 \geq 0$
 $x_2 \leq 2$



Original LP:
 $Z = 27.67$
 $x_1, x_2 = (2.33, 2.67)$

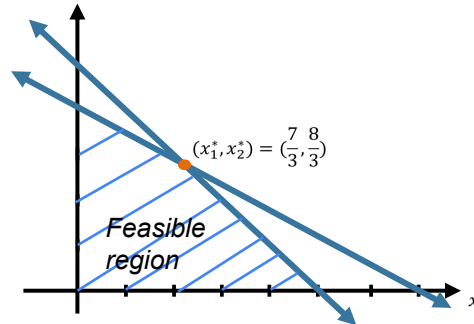
$$x_2 \leq 2$$



Branch and Bound Algorithm

Step 3: Check the neighboring integer solutions

Maximize $Z = 5x_1 + 6x_2$
Subject to $x_1 + x_2 \leq 5$
 $4x_1 + 7x_2 \leq 28$
 $x_1, x_2 \geq 0$
 $x_2 \leq 2$

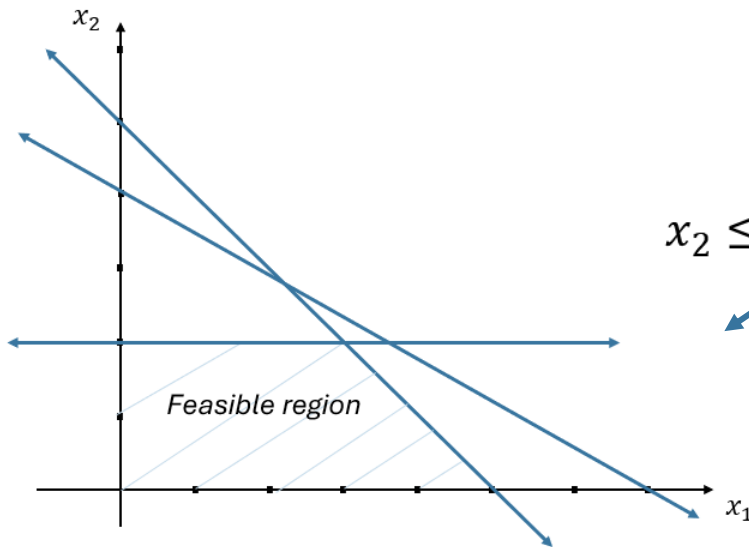
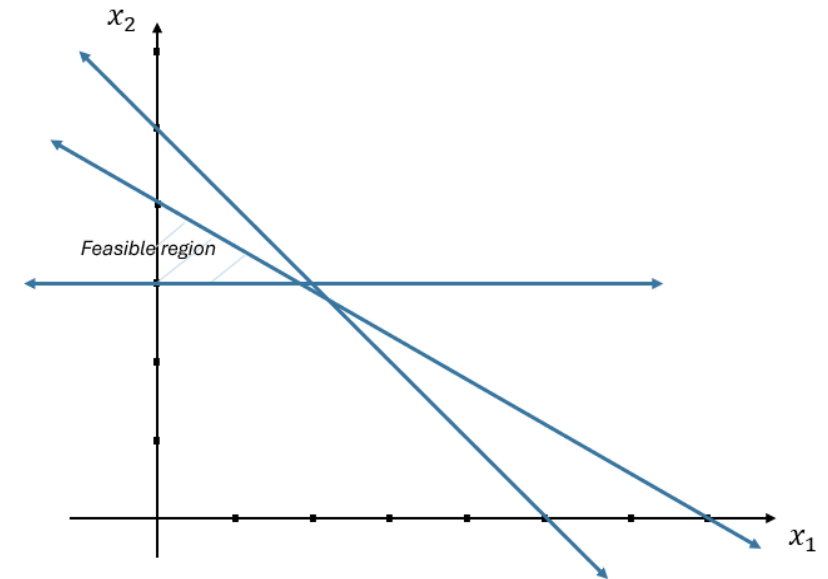


Original LP:
 $Z = 27.67$
 $x_1, x_2 = (2.33, 2.67)$

$x_2 \leq 2$

$x_2 \geq 3$

Maximize $Z = 5x_1 + 6x_2$
Subject to $x_1 + x_2 \leq 5$
 $4x_1 + 7x_2 \leq 28$
 $x_1, x_2 \geq 0$
 $x_2 \geq 3$



Branch and Bound Algorithm

Step 3: Check the neighboring integer solutions

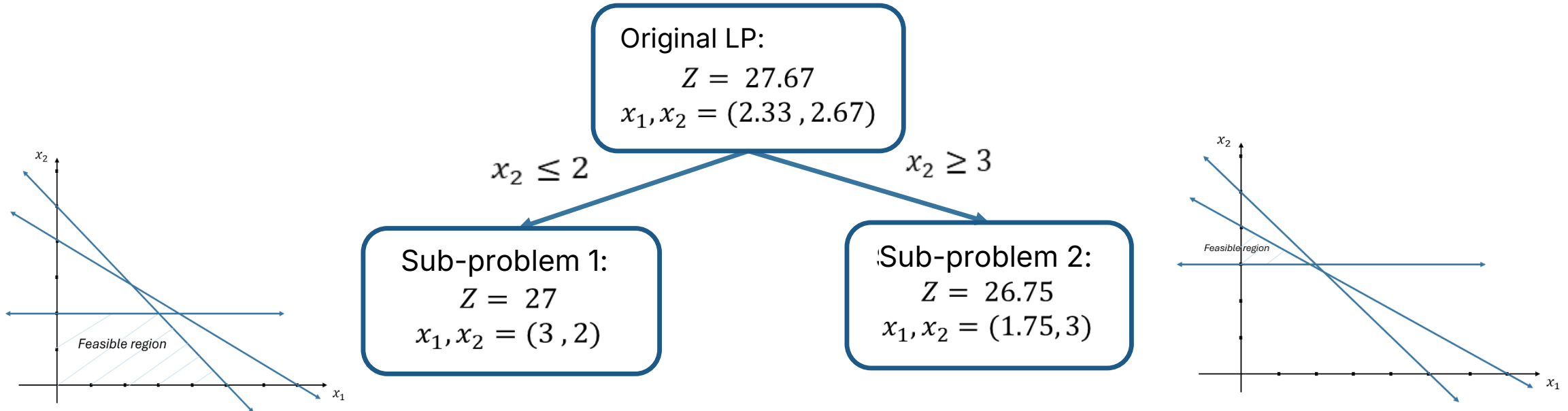
Original LP:
 $Z = 27.67$
 $x_1, x_2 = (2.33, 2.67)$

Maximize $Z = 5x_1 + 6x_2$
Subject to $x_1 + x_2 \leq 5$
 $4x_1 + 7x_2 \leq 28$
 $x_1, x_2 \geq 0$
 $x_2 \geq 3$

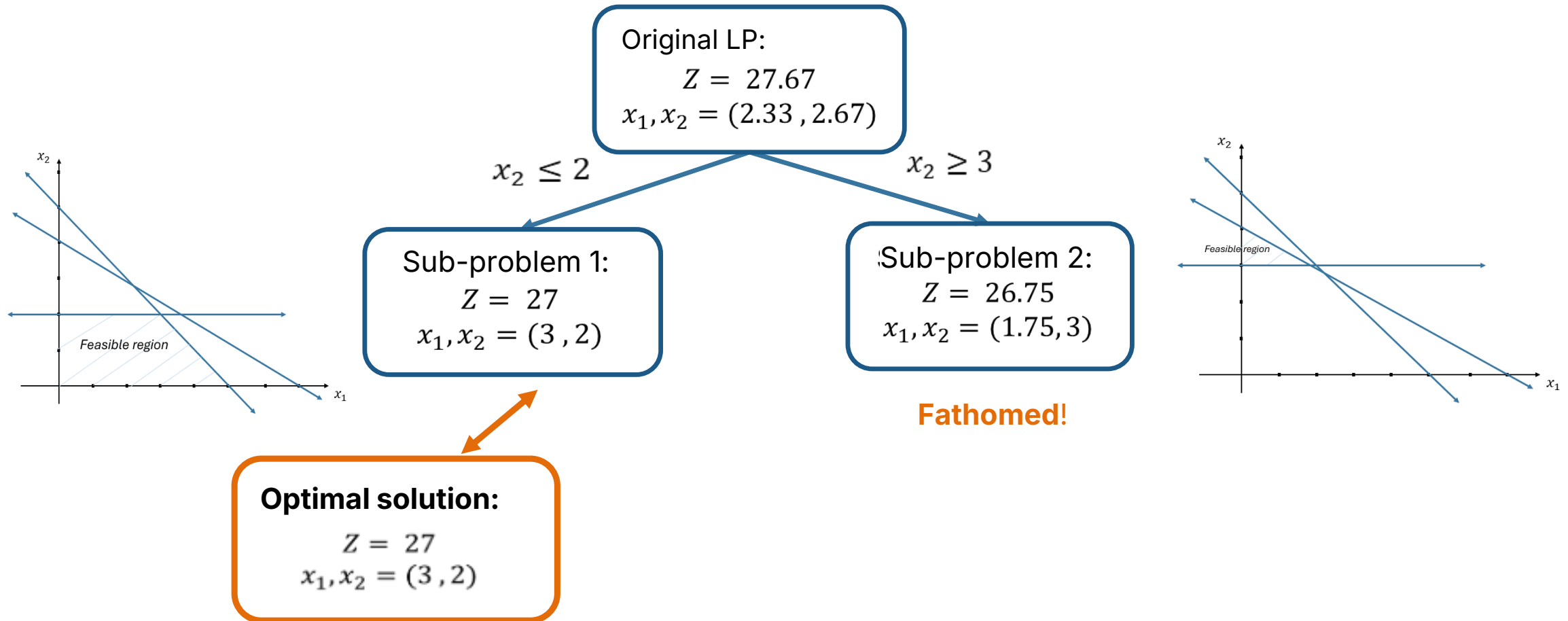
$x_2 \geq 3$

Sub-problem 2:
 $Z = 26.75$
 $x_1, x_2 = (1.75, 3)$

Branch and Bound Algorithm



Branch and Bound Algorithm



Branch and Bound Algorithm in a Nutshell

Suppose $\bar{x}$ is the integer solution of maximum profit found so far.

Iteratively consider a subproblem (node) of the branching tree:

Step 1: Solve the LR

Step 2:

- if x^* is integer and profit of x^* is larger than the profit of $\bar{x}$, prune the node and set $\bar{x} = x^*$
- if the profit of x^* is not larger than the profit of $\bar{x}$, prune the node
- if it is **infeasible**, **prune** the node
- else, let $x_i^* = \alpha \notin Z$. Create two subproblems adding $x_i \leq \lfloor \alpha \rfloor$ and $x_i \geq \lceil \alpha \rceil$ to the original IP.

What if the problem is Mixed-Integer?

$$\begin{array}{rclcl} \max & c^T x & + & d^T y & \\ & Ax & + & By & \leq b \\ & x & & & \in \mathbb{Z}^n \\ & & & y & \in \mathbb{R}^m \end{array}$$

Iteratively decomposes the original problem into two subproblems, storing in $(\bar{x}, \bar{y})$ the best feasible solution found so far.

For each of those subproblems:

- Solve the Linear Relaxation
- If the LR is infeasible, prune the node. Else, let (x^*, y^*) the optimal solution;
- If $x^* \in Z$ and $c^T x^* + d^T y^* > c^T \bar{x} + d^T \bar{y}$, prune the node and set $(\bar{x}, \bar{y}) = (x^*, y^*)$
- If $x^* \notin Z$. If $c^T x^* + d^T y^* \leq c^T \bar{x} + d^T \bar{y}$, prune the node.
- Else, let $x_i^* = \alpha \notin Z$. Create two subproblems adding $x_i \leq \lfloor \alpha \rfloor$ and $x_i \geq \lceil \alpha \rceil$

Pros and Cons of B&B



Usually good in practice.



In general, no guarantee on when it will terminate.

Cutting Planes Algorithm

Integer Programming Formulations

There are often many ways to formulate an IP

- Each way yields the exact same integer program

- The associated linear program relaxations, however, *may be very different*

 - They differ primarily in the bounds they yield

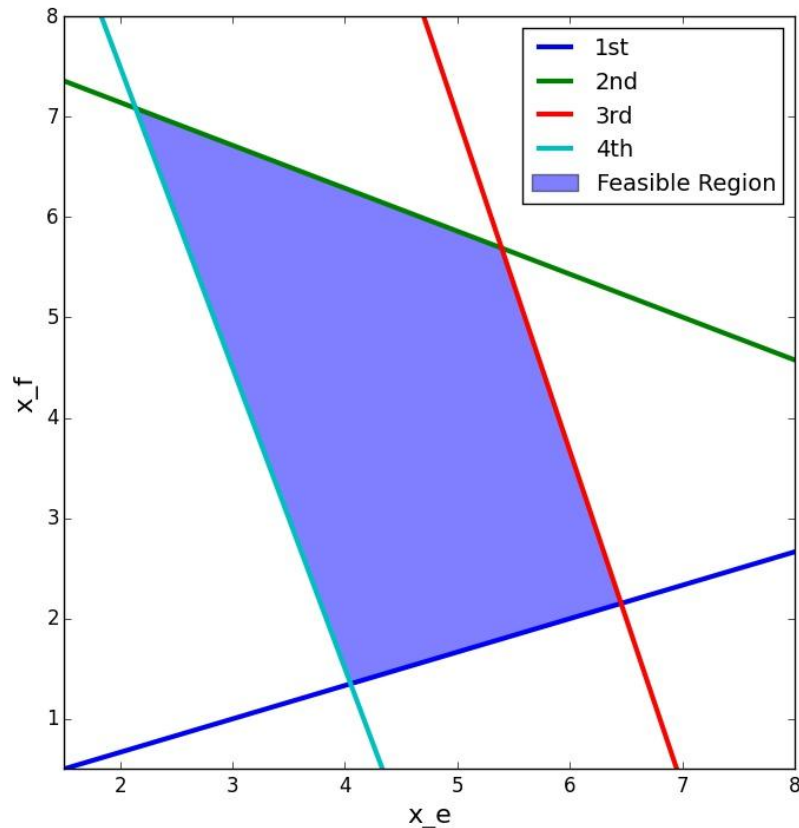
Illustrative IP: LP vs. IP Solutions

Let's consider this IP

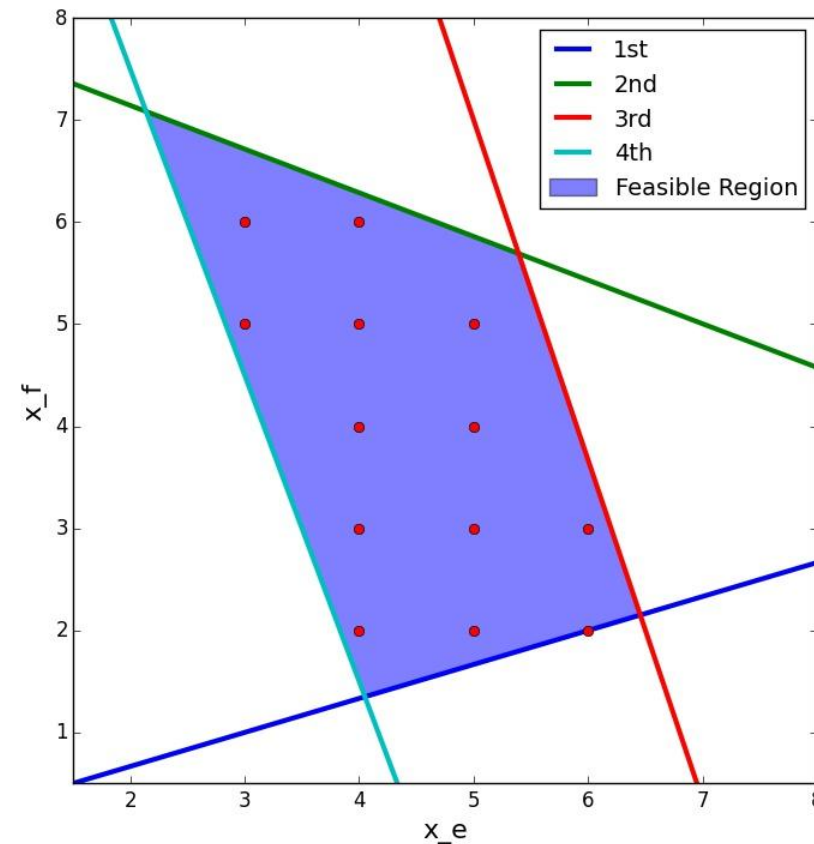
$$\begin{aligned} \max \quad & z = 18x_e + 6x_f \\ \text{subject to} \quad & x_e - 3x_f \leq 0 \\ & 42.8x_e + 100x_f \leq 800 \\ & 20x_e + 6x_f \leq 142 \\ & 30x_e + 10x_f \geq 135 \\ & x_e, x_f \geq 0, \text{ integer} \end{aligned}$$

Illustrative IP: LP vs. IP Solutions

It has a feasible region that appears roughly like this:



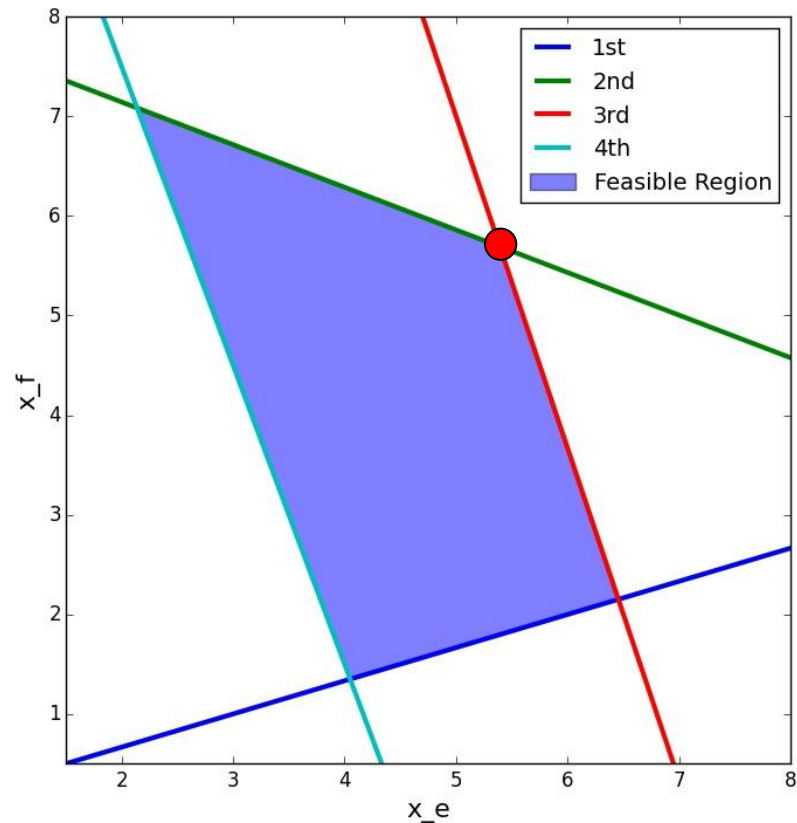
With integer variable restrictions, like this:



Illustrative IP: LP vs. IP Solutions

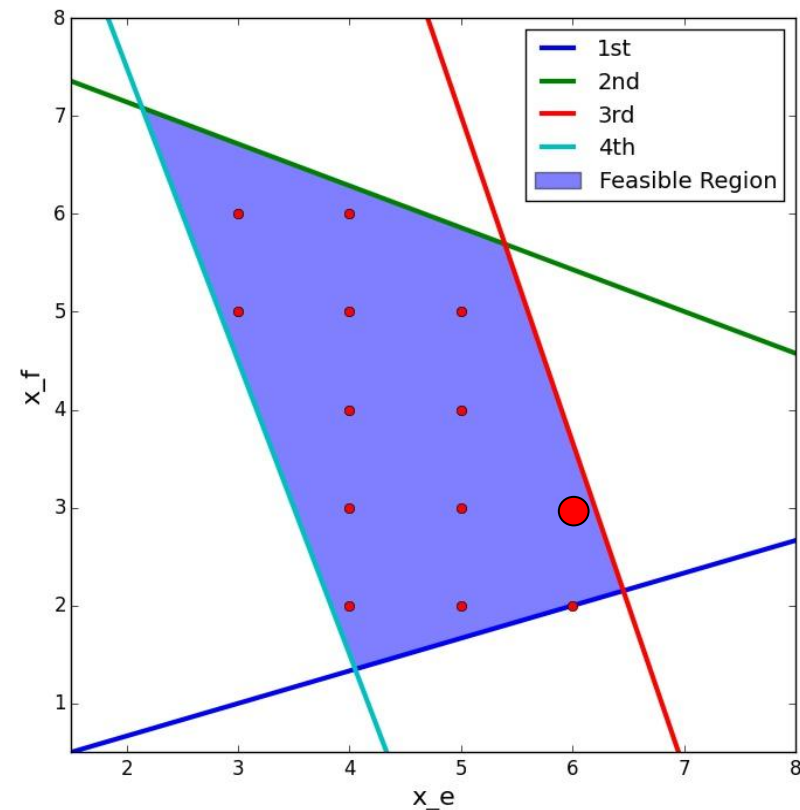
The optimal LP relaxation solution is:

$$z_{LP}^* = 131.2, x_e^* = 5.392, x_f^* = 5.692$$



The optimal integer solution is:

$$z_{IP}^* = 126, x_e^* = 6, x_f^* = 3$$



What's Best We Can Do?

We can add *cutting planes* that eliminate *fractional solutions* to the LP without eliminating any *integer solutions*

What's Best We Can Do?

We can add *cutting planes* that eliminate *fractional solutions* to the LP without eliminating any *integer solutions*

Adding the following 7 constraints, or *cutting planes*, to the problem yields the following:

$$x_f \geq 2$$

$$x_f \leq 6$$

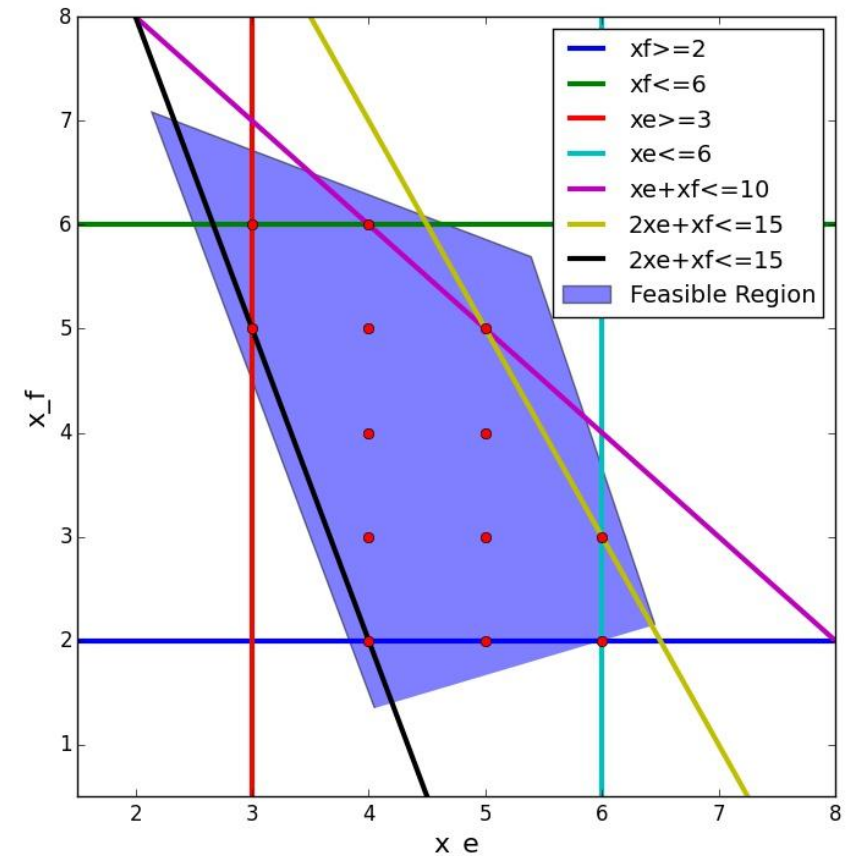
$$x_e \geq 3$$

$$x_e \leq 6$$

$$x_e + x_f \leq 10$$

$$2x_e + x_f \leq 15$$

$$3x_e + x_f \geq 14$$



What's Best We Can Do?

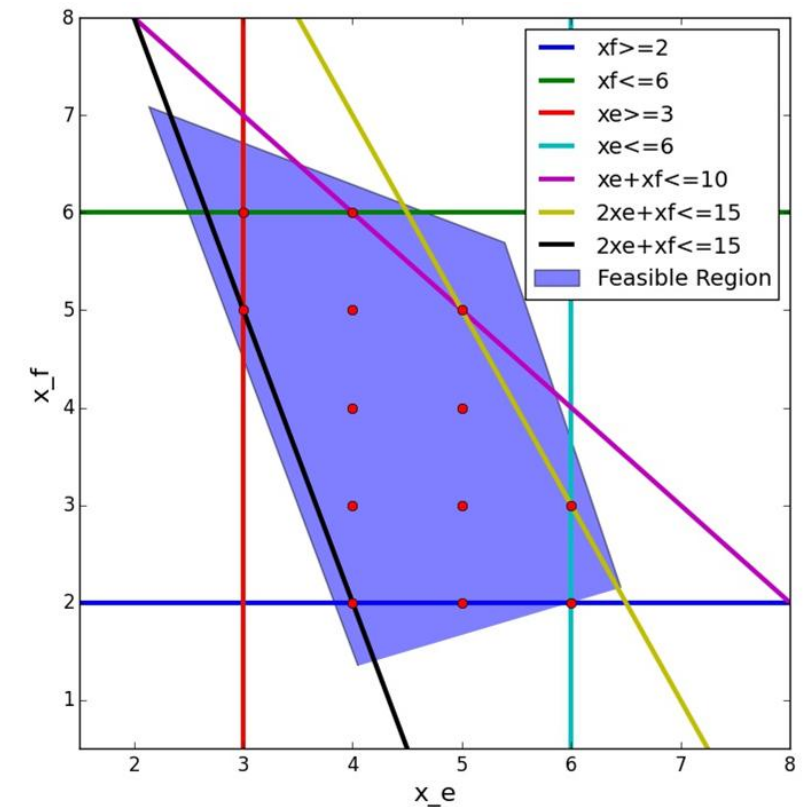
Adding these seven *valid inequalities* causes every corner point of the LP to be integer

Thus, the IP can be solved by solving the LP relaxation

What's Best We Can Do?

Adding these seven *valid inequalities* causes every corner point of the LP to be integer

Thus, the IP can be solved by solving the LP relaxation



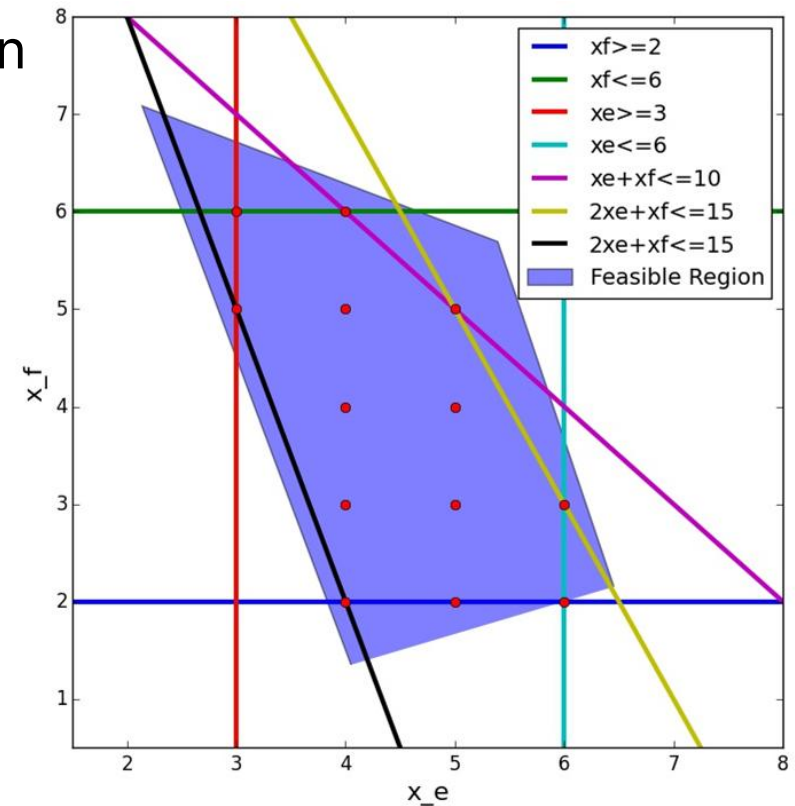
What's Best We Can Do?

Adding these seven *valid inequalities* causes every corner point of the LP to be integer

Thus, the IP can be solved by solving the LP relaxation

These 7 cutting planes form the ***convex hull of integer solutions***

- The deepest cuts are called ***facets***
- We obtained it by inspection, but will discuss some general ways to obtain it later in the lecture
- *Such constraints are difficult to find for general (and especially, large) IPs*



How to Find Such Valid Inequalities?

There are approaches to find valid, or even facet-defining, inequalities for integer programs

- Some are problem specific
- Others are general to any integer program

These valid inequalities are typically added during the branch and bound process

- Known as *branch and cut*

General Valid Inequalities

The motivation is rather straightforward

- Consider the following generic integer program:

$$\begin{array}{ll}\max & c^\top x \\ \text{subject to} & Ax \leq b \\ & x \geq 0, \text{ integer}\end{array}$$

General Valid Inequalities

The motivation is rather straightforward

- Consider the following generic integer program:

$$\begin{array}{ll}\max & c^\top x \\ \text{subject to} & Ax \leq b \\ & x \geq 0, \text{ integer}\end{array}$$

- Now suppose we have any $u \in \mathbb{R}_+^m$
- Then it is entirely valid to write: $(u^\top A)x \leq u^\top b$
 - Right?

General Valid Inequalities

- Observe that our original variables are nonnegative: $x \geq 0$
- Then the following operation to the left-hand side of the relationship is valid to do: $(\lfloor u^\top A \rfloor)x \leq u^\top b$
 - *Why is this?*

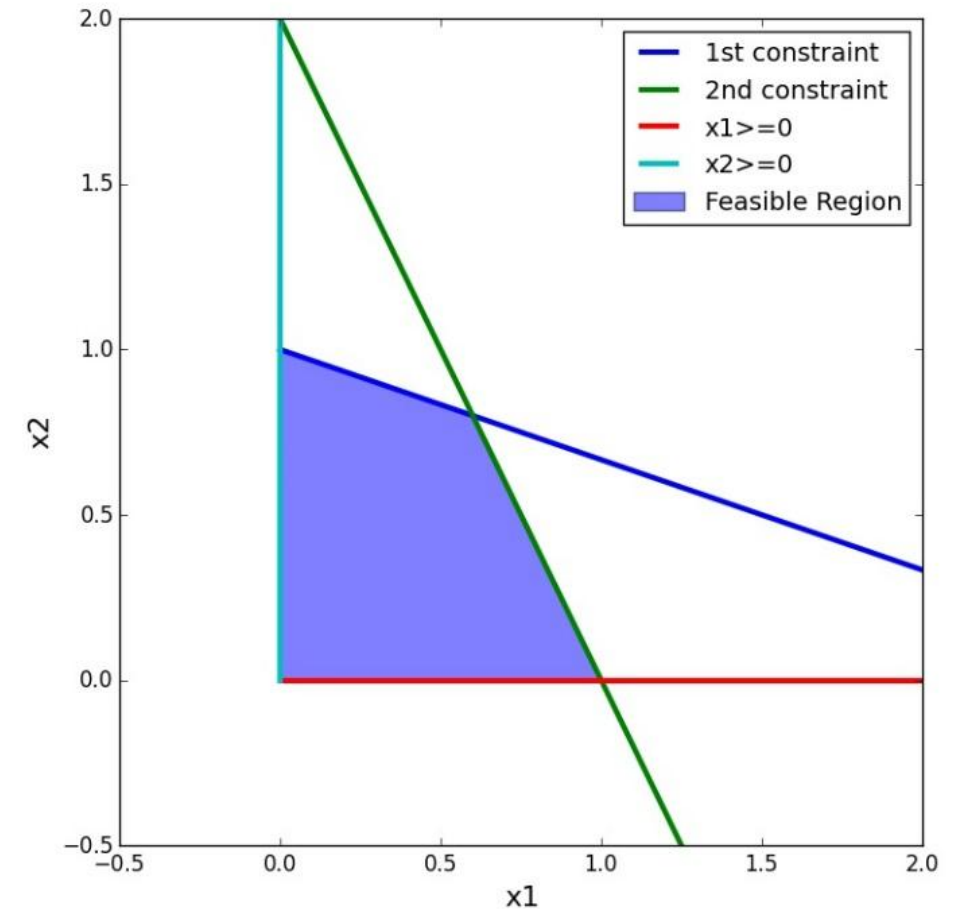
General Valid Inequalities

- Observe that our original variables are nonnegative: $x \geq 0$
- Then the following operation to the left-hand side of the relationship is valid to do: $(\lfloor u^\top A \rfloor)x \leq u^\top b$
 - *Why is this?*
- **Here's the key:**
 - Because of the integrality of both $\lfloor u^\top A \rfloor$ and x
 - We can now write: $(\lfloor u^\top A \rfloor)x \leq \lfloor u^\top b \rfloor$
 - This inequality is known as a **Chvátal-Gomory (CG)** cut
(*The ability to round down the right-hand side is very important*)

Example: Chvátal-Gomory Cut

- Suppose we have the following system:

$$\begin{array}{ll}\max & 5x_1 + 4x_2 \\ \text{subject to} & x_1 + 3x_2 \leq 3 \\ & 2x_1 + x_2 \leq 2 \\ & x_1, x_2 \geq 0, \text{ integer}\end{array}$$



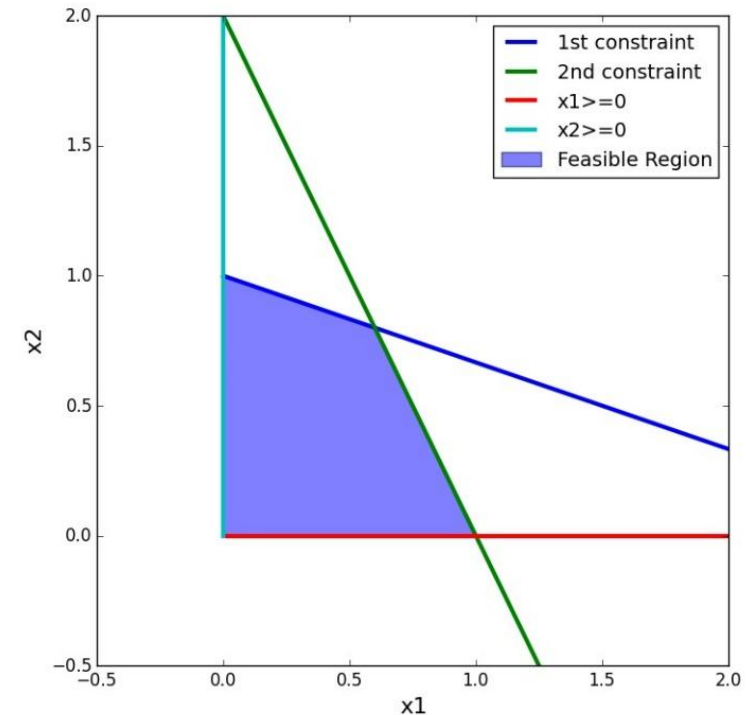
Example: Chvátal-Gomory Cut

- Suppose we have the following system:

$$\begin{aligned} \max \quad & 5x_1 + 4x_2 \\ \text{subject to} \quad & x_1 + 3x_2 \leq 3 \\ & 2x_1 + x_2 \leq 2 \\ & x_1, x_2 \geq 0, \text{ integer} \end{aligned}$$

- Here are the important components

- In matrix / vector form: $A = \begin{bmatrix} 1 & 3 \\ 2 & 1 \end{bmatrix} \quad b = \begin{bmatrix} 3 \\ 2 \end{bmatrix}$



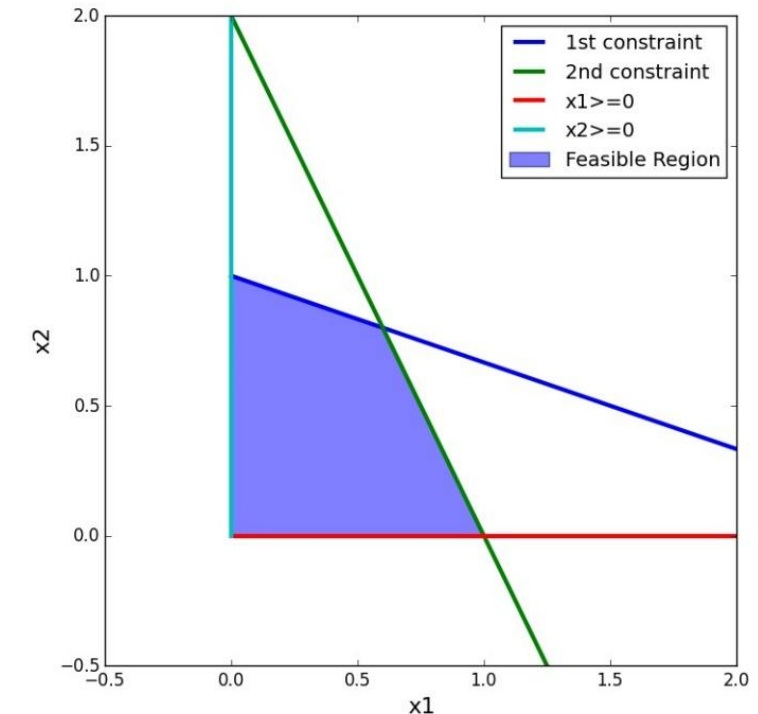
Example: Chvátal-Gomory Cut

- Suppose we have the following system:

$$\begin{aligned} \max \quad & 5x_1 + 4x_2 \\ \text{subject to} \quad & x_1 + 3x_2 \leq 3 \\ & 2x_1 + x_2 \leq 2 \\ & x_1, x_2 \geq 0, \text{ integer} \end{aligned}$$

- Here are the important components

- In matrix / vector form: $A = \begin{bmatrix} 1 & 3 \\ 2 & 1 \end{bmatrix} \quad b = \begin{bmatrix} 3 \\ 2 \end{bmatrix}$



- Now suppose we use this (arbitrary) vector: $u^T = \begin{bmatrix} 1 \\ 5 \end{bmatrix}, \begin{bmatrix} 2 \\ 5 \end{bmatrix}$

Example: Chvátal-Gomory Cut

- The operation $\lfloor u^\top A \rfloor$ yields $\lfloor 1, 1 \rfloor = [1, 1]$
- The operation $\lfloor u^\top b \rfloor$ yields $\lfloor 1.4 \rfloor = 1$

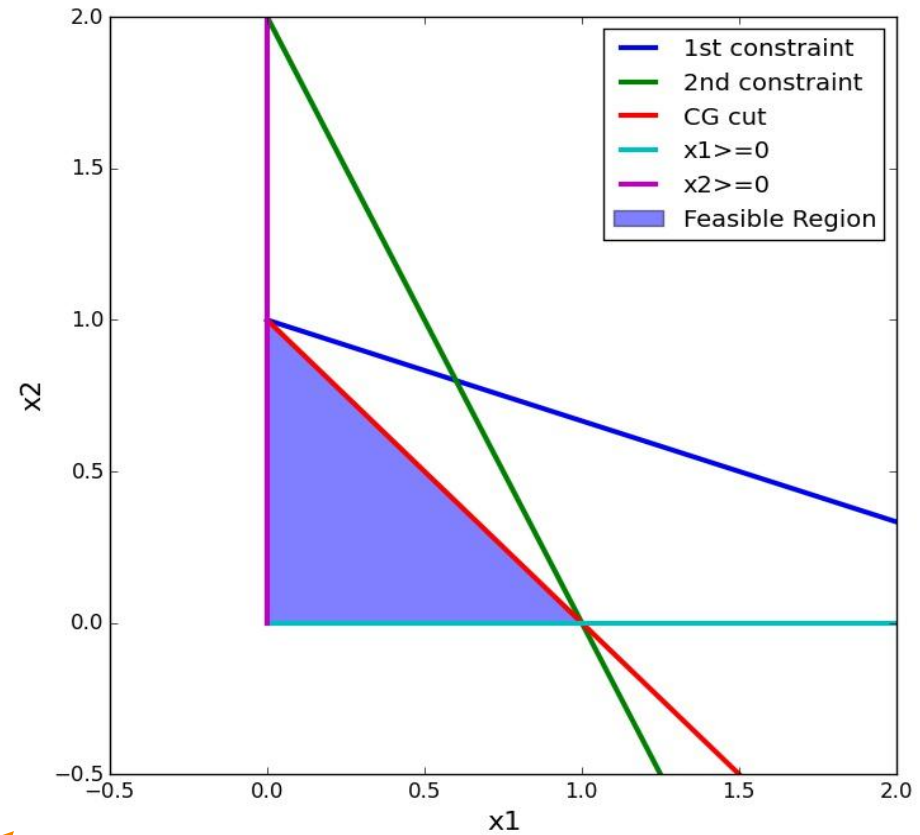
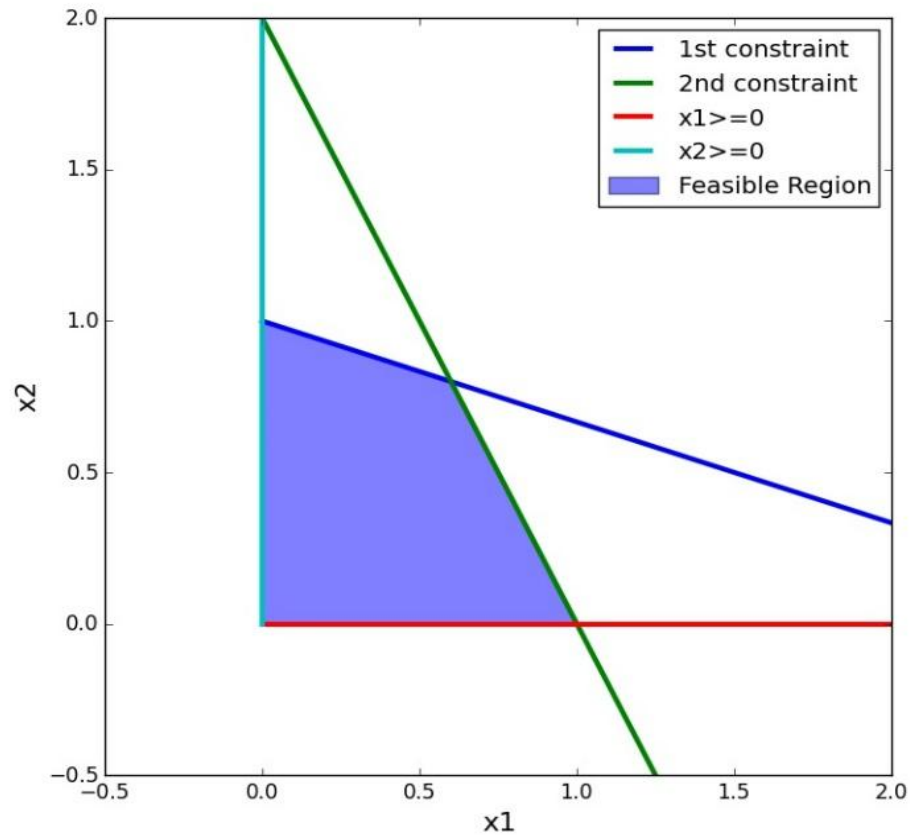
Example: Chvátal-Gomory Cut

- The operation $\lfloor u^\top A \rfloor$ yields $\lfloor 1, 1 \rfloor = [1, 1]$
- The operation $\lfloor u^\top b \rfloor$ yields $\lfloor 1.4 \rfloor = 1$

So, this generates the new Chvátal-Gomory (CG) cut: $x_1 + x_2 \leq 1$

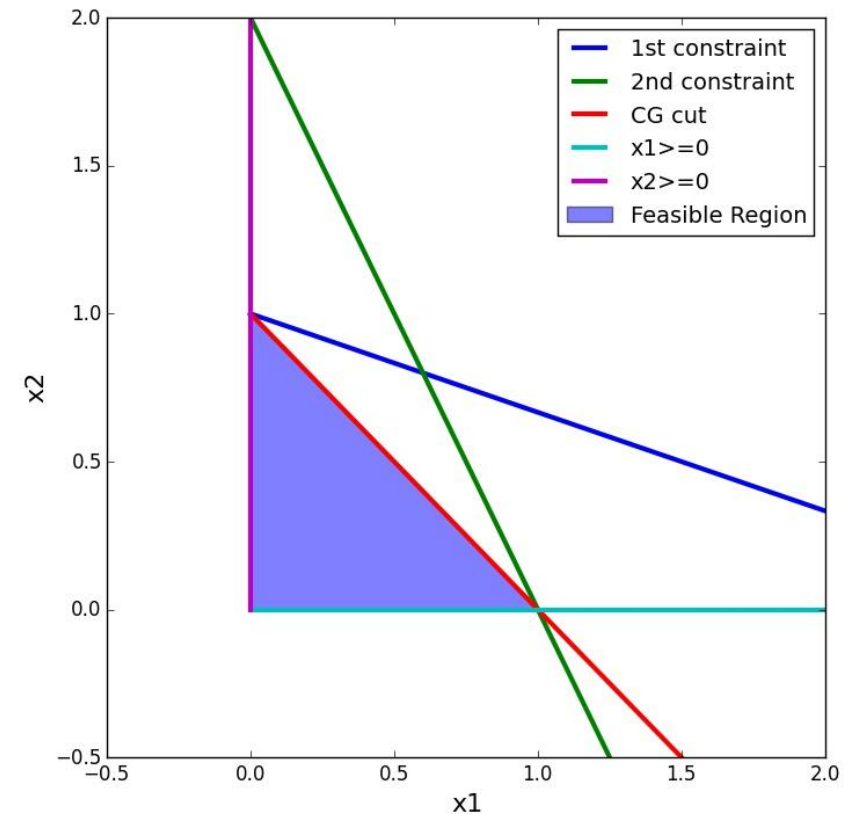
Example: Chvátal-Gomory Cut

So, this generates the new Chvátal-Gomory (CG) cut: $x_1 + x_2 \leq 1$



Example: Chvátal-Gomory Cut

- In this case, this operation yields the convex hull of feasible integer solutions
- In general, it's not obvious how to choose the vector u
- Can iteratively apply multiple CG cuts to remove fractional space



Ralph Gomory And IP

<https://www.youtube.com/watch?v=OesVp4Hlqps>



Ralph Gomory. Source: *The Heinz Awards*. heinzawards.net
(accessed August 7, 2025)

Cutting Plane Approach In A Nutshell



Another approach to solving integer programs is the cutting plane approach

- Pioneered by Ralph Gomory in the late 1950's

Cutting Plane Approach In A Nutshell



Another approach to solving integer programs is the cutting plane approach

- Pioneered by Ralph Gomory in the late 1950's

Just a brief overview of the main idea:

- First, solve the LP relaxation
- If the solution is non-integer, generate some kind of **valid inequality** that: Cuts off the fractional solution, and does not cut off any integer solution
- Repeat process until an integer solution is found

Cutting Plane Approach In A Nutshell



Another approach to solving integer programs is the cutting plane approach

- Pioneered by Ralph Gomory in the late 1950's

Just a brief overview of the main idea:

- First, solve the LP relaxation
- If the solution is non-integer, generate some kind of **valid inequality** that: Cuts off the fractional solution, and does not cut off any integer solution
- Repeat process until an integer solution is found

Sometimes valid inequalities, or cuts, are added dynamically during the branch and bound process: this is known as the **branch-and-cut algorithm**

Implementation of Algorithms Using Gurobi

Example: A Clothing Company



A Clothing Company is capable of manufacturing three types of clothing: shirts, shorts, and pants.

The manufacture of each type of clothing requires that company have the appropriate type of machinery available. The machinery needed to manufacture each type of clothing must be rented at the following rates:

- shirt machinery: \$200 per week
- shorts machinery: \$150 per week
- pants machinery: \$100 per week

It also requires the amounts of cloth and labor as shown in Table 1. Each week, 150 hours of labor and 160 sq yd of cloth are available. The variable unit cost and selling price for each type of clothing are shown in Table 2. Formulate an IP whose solution will maximize company's weekly profits.

Resource Requirements

Clothing Type	Labor (Hours)	Cloth (Square Yards)
Shirt	3	4
Shorts	2	3
Pants	6	4

Revenue and Cost Information

Clothing Type	Sales Price (\$)	Variable Cost (\$)
Shirt	12	6
Shorts	8	4
Pants	15	8

Example: A Clothing Company



Define the decision variables:

x_1 = number of shirts produced each week

x_2 = number of shorts produced each week

x_3 = number of pants produced each week

$$y_1 = \begin{cases} 1 & \text{if any shirts are manufactured} \\ 0 & \text{otherwise} \end{cases}$$

$$y_2 = \begin{cases} 1 & \text{if any shorts are manufactured} \\ 0 & \text{otherwise} \end{cases}$$

$$y_3 = \begin{cases} 1 & \text{if any pants are manufactured} \\ 0 & \text{otherwise} \end{cases}$$

In short, if $x_j > 0$, then $y_j = 1$, and if $x_j = 0$, then $y_j = 0$.

Example: A Clothing Company



Define the objective function:

Company's Weekly Profits:

(weekly sales revenue) - (weekly variable costs) - (weekly costs of renting machinery)

$$\begin{aligned}\text{Weekly profit} &= (12x_1 + 8x_2 + 15x_3) - (6x_1 + 4x_2 + 8x_3) \\ &\quad - (200y_1 + 150y_2 + 100y_3) \\ &= 6x_1 + 4x_2 + 7x_3 - 200y_1 - 150y_2 - 100y_3\end{aligned}$$

$$\text{Maximize } z = 6x_1 + 4x_2 + 7x_3 - 200y_1 - 150y_2 - 100y_3$$

Example: A Clothing Company



Then the IP becomes:

$$\max z = 6x_1 + 4x_2 + 7x_3 - 200y_1 - 150y_2 - 100y_3$$

$$\text{s.t.} \quad 3x_1 + 2x_2 + 6x_3 \leq 150$$

$$4x_1 + 3x_2 + 4x_3 \leq 160$$

$$x_1, x_2, x_3 \geq 0; x_1, x_2, x_3 \text{ integer}$$

$$y_1, y_2, y_3 = 0 \text{ or } 1$$

$$x_1 \leq M_1y_1$$

$$x_2 \leq M_2y_2$$

$$x_3 \leq M_3y_3$$

Common IP Problems: The Traveling Salesperson Problem

- IP Formulation and the Problem of Subtour Growth
- Miller, Tucker, Zemlin Formulation

Traveling Salesperson Problem

Perhaps the most classical of optimization problems

- A salesperson needs to design a route ("tour") through n cities, returning to their original location
 - Would like to do so in the shortest possible distance
- Wide variety of applications:
 - Routing and Logistics
 - Integrated circuits design
 - Imaging of celestial* objects
 - Artwork!

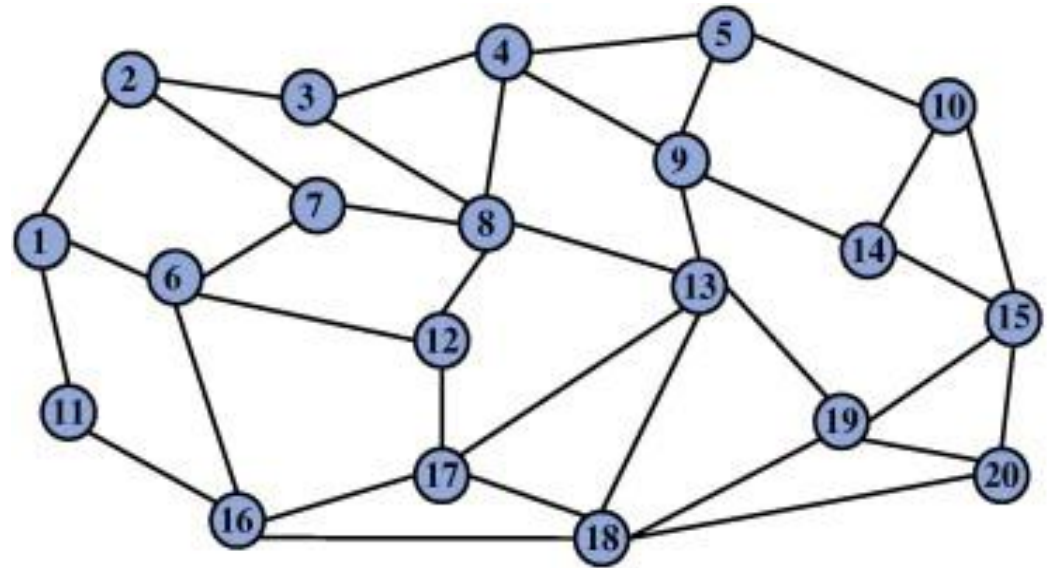


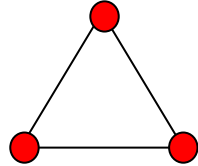
Figure 2. Structure of the emergency logistics network. Adapted from "Route selection for Emergency Logistics Management: A bio-inspired algorithm," by X. Zhang, Z. Zhang, Y. Zhang, D. Wei, and Y. Deng, 2013, *Safety Science*, 54, p. 89 (<https://doi.org/10.1016/j.ssci.2012.12.003>). Copyright 2013 by Elsevier.

*astronomical

How Hard Could That Be?

What if there just three cities ... how many unique ways to find a circuit through them?

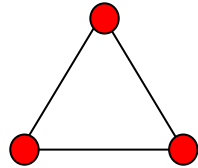
- Just one way!



How Hard Could That Be?

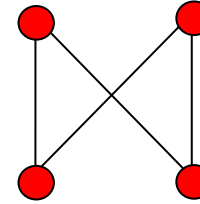
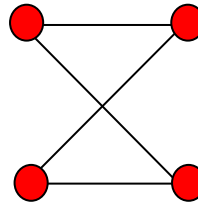
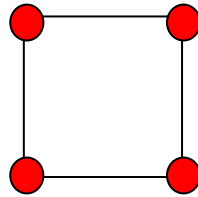
What if there just three cities ... how many unique ways to find a circuit through them?

- Just one way!



What about four cities?

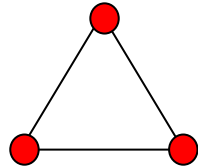
... There are 3 ways



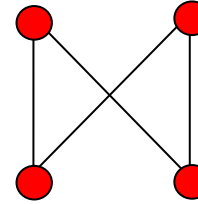
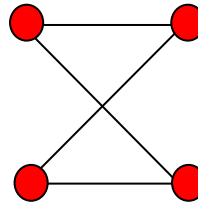
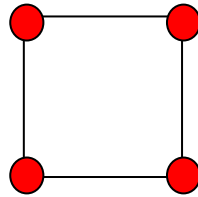
How Hard Could That Be?

What if there just three cities ... how many unique ways to find a circuit through them?

- Just one way!



What about four cities?
... There are 3 ways



And five cities? ...Twelve ways

Six cities? Eight? Ten? Twelve? Twenty?

60; 20,160; 1,814,400; 239,500,800; 1,216,451,004,088,320,000 ways

From A Procter & Gamble Ad - 1962:

Imagine that Toody and Muldoon want to drive around the country and visit each of the 33 locations represented by dots on the contest map, and that in doing so, they want to travel the shortest possible route. You should plan a route for them from location to location which will result in the shortest total mileage from Chicago, Illinois back to Chicago, Illinois.

10,000 \$ in 1962 correspond to \$80,000 in today's value.



Source: University of Houston – *The Engines of Our Ingenuity*

Let's try solving the TSP together!

Navigate here:

https://algorithms.discrete.ma.tum.de/graph-games/tsp-game/index_en.html

Choose Deutschland (Germany) as your map

Number of cities: 25

Use Euclidean distance

Other Facts About The TSP

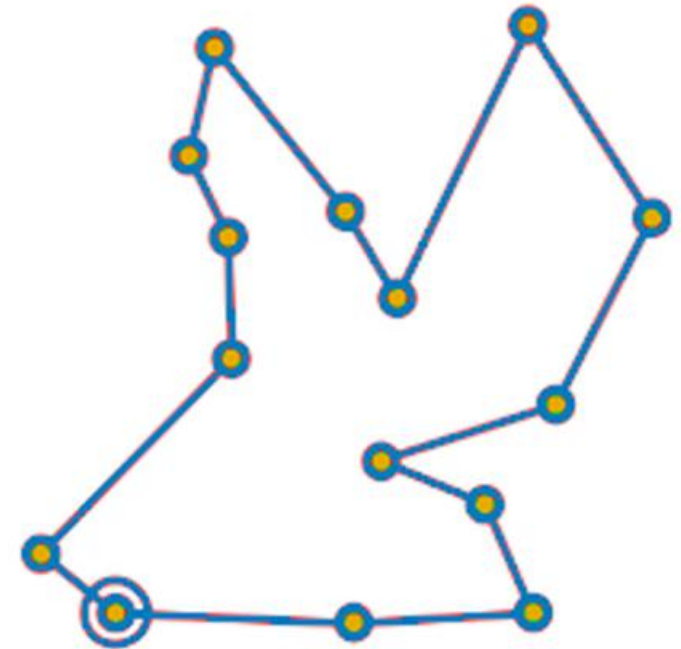
- This problem has been studied very well
- It is NP-hard (Non-deterministic Polynomial-time hard) *difficult to solve, computationally speaking*
- That said, large instances have been solved optimally (over 30,000 "cities" – actually, a microchip layout problem)
- Very large instances have been solved approximately (ten million cities to within 1 or so percent of optimum)

How To Solve TSP Computationally?

While the basic TSP resembles a totally connected network, unfortunately it does not share the beneficial integrality properties of other network flow problems

- Heuristic approaches exist:
 - Find good solutions
 - No guarantee of quality

You are within 1.0501% of the optimum.



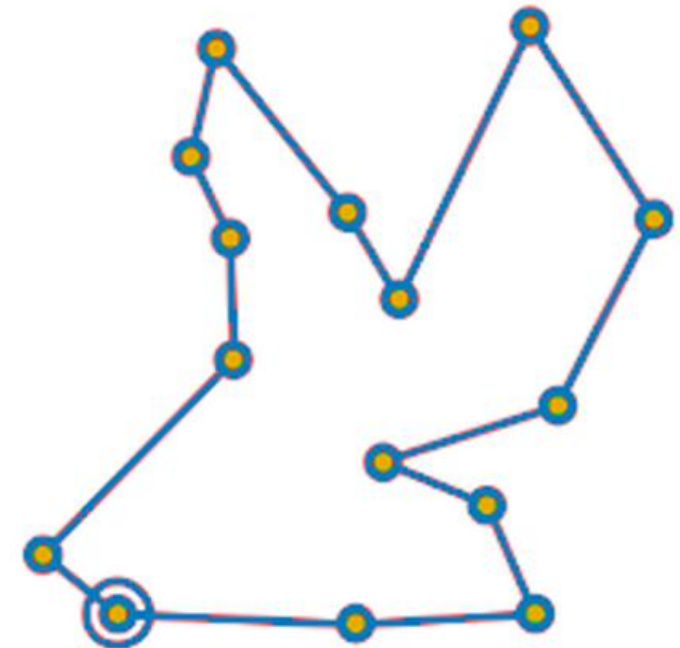
Cities: 15
Length: 1,251.00

How To Solve TSP Computationally?

While the basic TSP resembles a totally connected network, unfortunately it does not share the beneficial integrality properties of other network flow problems

- Heuristic approaches exist:
 - Find good solutions
 - No guarantee of quality
- We will approach it as a binary integer program
 - Guarantee on global optimality
 - Multiple ways to do this
 - We consider a *symmetric* TSP formulation, where the distance from i to j , is the same from j to i

You are within 1.0501% of the optimum.



Cities: 15
Length: 1,251.00

Formulating The TSP (Symmetric)

This can be done completely with binary variables!

- Suppose N is a set of cities, indexed by i and j .
- Let d_{ij} be the distance from city i to city j , $i \neq j$.
- Let $x_{ij} = \begin{cases} 1 & \text{if arc from node } i \text{ to node } j \text{ is traversed, } i \neq j; \\ 0 & \text{otherwise.} \end{cases}$

$$\begin{array}{ll} \min & \sum_{i \in N} \sum_{j \in N: j \neq i} d_{ij} x_{ij} \\ \text{subject to} & \sum_{i \in N: i \neq j} x_{ij} = 1 \quad \forall j \in N, \\ & \sum_{j \in N: j \neq i} x_{ij} = 1 \quad \forall i \in N, \\ & x_{ij} \in \{0, 1\} \quad \forall i \in N, \quad \forall j \in N. \end{array}$$

Formulating The TSP (Symmetric)

This can be done completely with binary variables!

- Suppose N is a set of cities, indexed by i and j .
- Let d_{ij} be the distance from city i to city j , $i \neq j$.
- Let $x_{ij} = \begin{cases} 1 & \text{if arc from node } i \text{ to node } j \text{ is traversed, } i \neq j; \\ 0 & \text{otherwise.} \end{cases}$

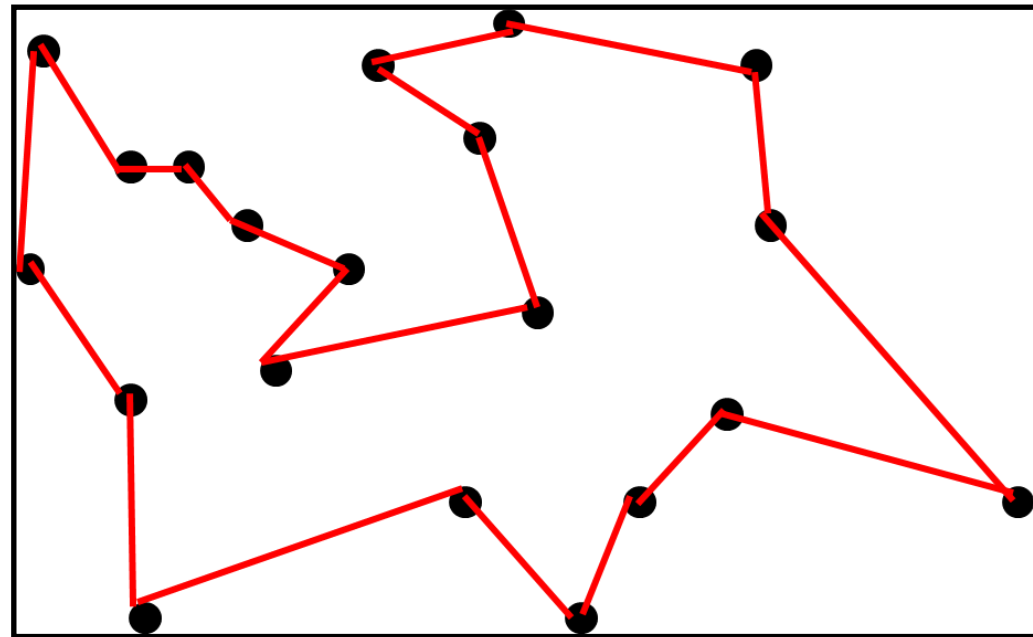
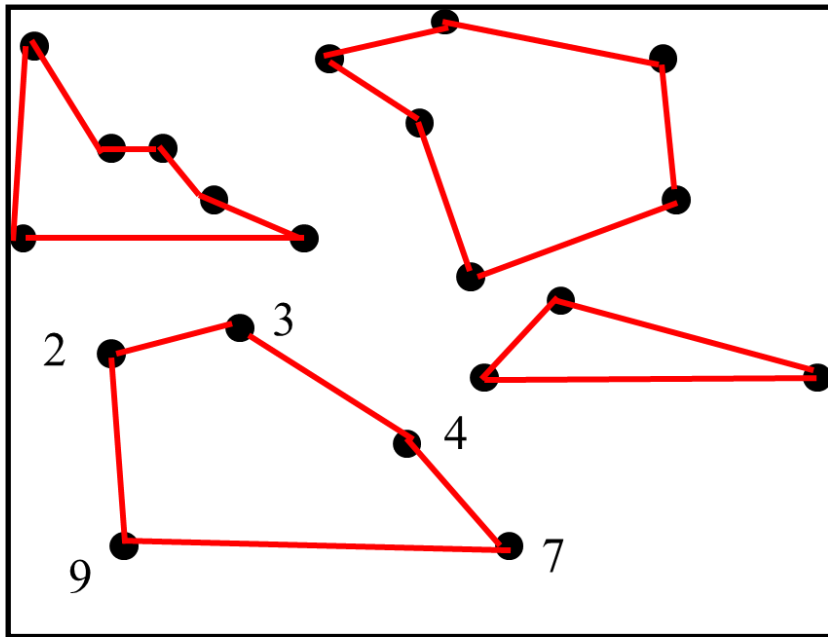
$$\begin{array}{ll} \min & \sum_{i \in N} \sum_{j \in N: j \neq i} d_{ij} x_{ij} \\ \text{subject to} & \sum_{i \in N: i \neq j} x_{ij} = 1 \quad \forall j \in N, \\ & \sum_{j \in N: j \neq i} x_{ij} = 1 \quad \forall i \in N, \\ & x_{ij} \in \{0, 1\} \quad \forall i \in N, \quad \forall j \in N. \end{array}$$

Well, almost ...
something is still
missing ...
Anyone see it?

Subtours

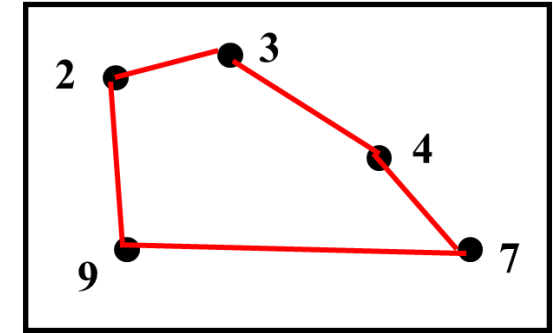
Subtours!

- The previous formulation may permit *subtours*:
Smaller tours that connect a strict subset of cities



A Subtour-Breaking Constraint

- Let S be any *proper* subset of nodes, $|S| \geq 3$
 - For example, $S = \{2, 3, 4, 7, 9\}$
- Two key observations:
 - A subtour that includes all nodes of S has $|S|$ arcs
 - A tour for the entire (sub)network of S *cannot contain more than* $|S|$
 - 1 arcs having two endpoints in S

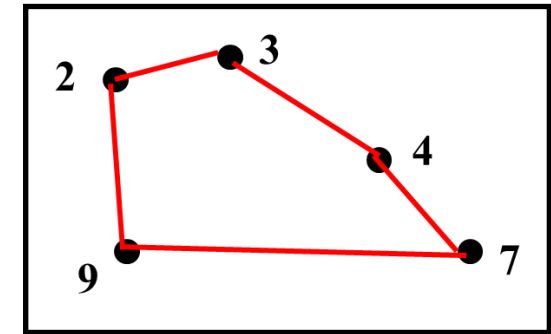


A Subtour-Breaking Constraint

- Let S be any *proper* subset of nodes, $|S| \geq 3$
 - For example, $S = \{2, 3, 4, 7, 9\}$
- Two key observations:
 - A subtour that includes all nodes of S has $|S|$ arcs
 - A tour for the entire (sub)network of S *cannot contain more than* $|S| - 1$ arcs *having two endpoints in* S
- To forbid a subtour on subset S of nodes, add constraint:

$$\sum_{i \in S} \sum_{j \in S; j \neq i} x_{ij} \leq |S| - 1$$

- This constraint sums ***all*** arcs in the subset S (i.e., not just the active ones)

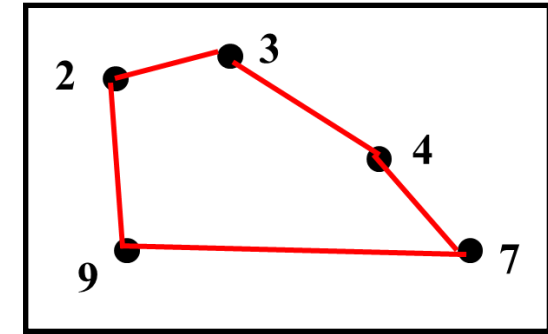


A Subtour-Breaking Constraint

- Let S be any *proper* subset of nodes, $|S| \geq 3$
 - For example, $S = \{2, 3, 4, 7, 9\}$
- Two key observations:
 - A subtour that includes all nodes of S has $|S|$ arcs
 - A tour for the entire (sub)network of S *cannot contain more than* $|S| - 1$ arcs having two endpoints in S
- To forbid a subtour on subset S of nodes, add constraint:

$$\sum_{i \in S} \sum_{j \in S; j \neq i} x_{ij} \leq |S| - 1$$

- This constraint sums **all** arcs in the subset S (i.e., not just the active ones)



Easy enough, right? How many possible subtours exist?

Subtour Growth!

- Adding all subtour constraints is not a problem for small networks, say, with 6 to 10 nodes (cities)
 - However, the number of subtours grows exponentially with the number of cities
 - Prohibitive to add all of them
 - Any ideas on what to do?

Subtour Growth!

- Adding all subtour constraints is not a problem for small networks, say, with 6 to 10 nodes (cities)
 - However, the number of subtours grows exponentially with the number of cities
 - Prohibitive to add all of them
 - Any ideas on what to do?
1. Include only some of the constraints, and solve
 2. If a subtour appears as part of the solution, add a new subtour elimination constraint and solve again